UNDERSTANDING

ALGORITHM

A Brief Overview of Program and Analysis of Algorithm

PRESENTED BY

MD. TANVIR RAHMAN PRANTO

JUNIOR RED TE4M AN4LY5T

CSE -26

0712340012101005

North Bengal International University

LINEAR SEARCH

Linear Search is the simplest searching algorithm used to find a particular element in a list or array. It works by checking each element one by one from the beginning until the desired value is found or the list ends.

Its also called Sequential Search Algorithm

ALGORITHM

Start from the first element of the array.

Compare the current element with the target value.

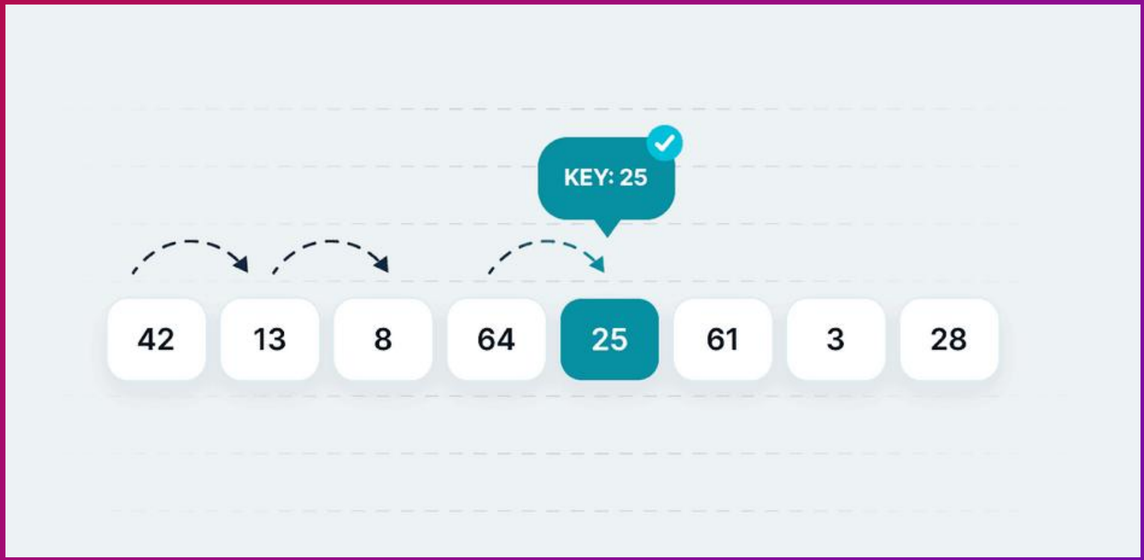
If it matches, return the index (or position).

If it doesn’t match, move to the next element.

Repeat steps 2–4 until the end of the array.

If the target is not found, return a Not found

LINEAR SEARCH



PSEUDOCODE :

```
START
READ n
READ array[0..n-1]
READ key
FOR i = 0 TO n-1
IF array[i] == key THEN
PRINT "Found at position", i+1
STOP
ENDIF
NEXT i
PRINT "Not found"
END
```

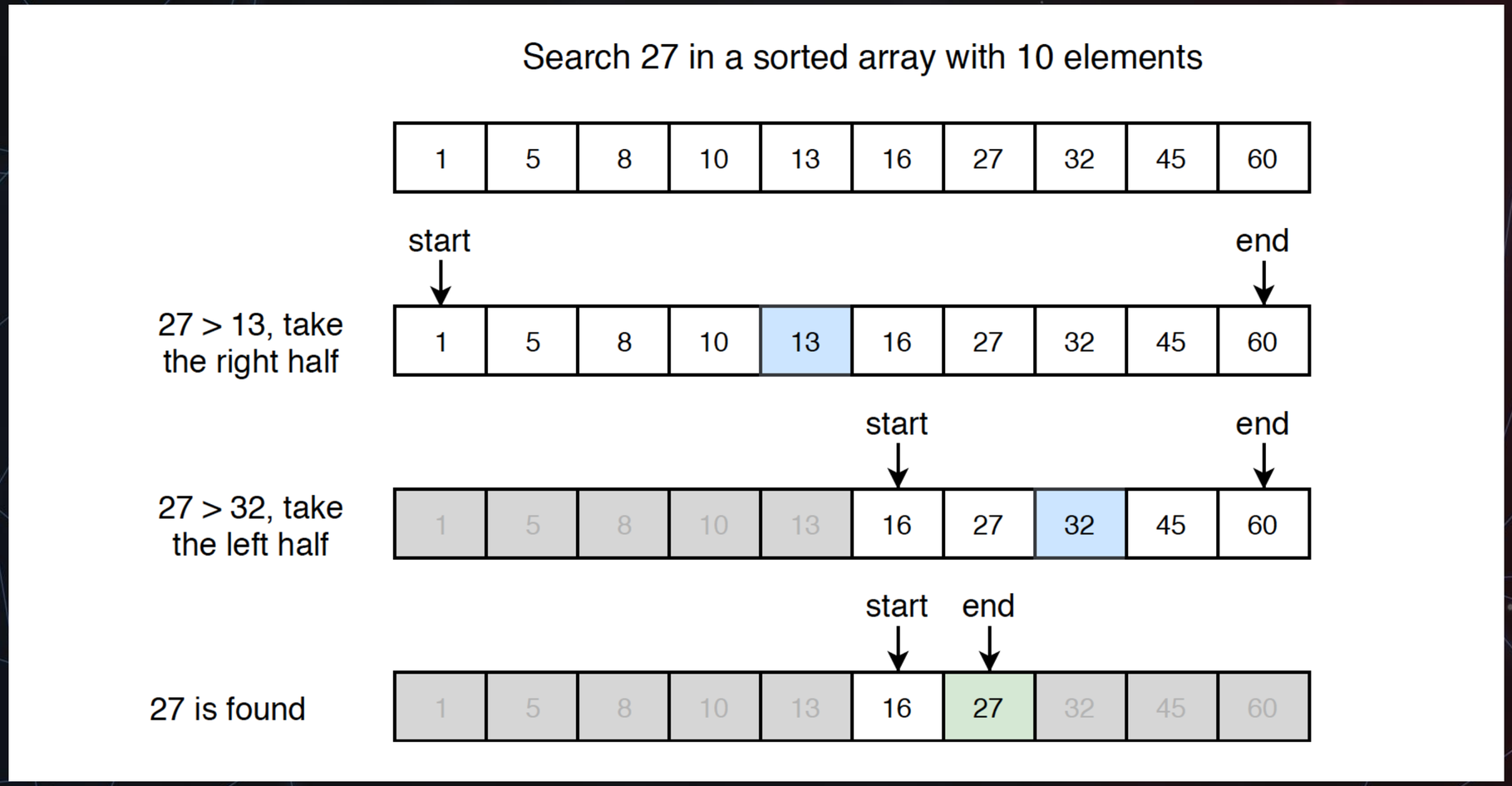
BINARY SEARCH

Binary search is a fast way to find a value in a sorted array (ascending order). It repeatedly halves the search range by comparing the middle element with the target.

ALGORITHM

- Read n (size of array) and the n elements in ascending order.
- Read the target value x.
- Set low = 0, high = n - 1.
- While low <= high:
  - Compute mid = low + (high - low) / 2.
  - If A[mid] == x: found → stop.
  - Else if A[mid] < x: search right half → low = mid + 1.
  - Else (A[mid] > x): search left half → high = mid - 1.
- If the loop finishes without finding, x is not found.

BINARY SEARCH



BUBBLE SORT

Bubble sort is a simple, comparison-based, in-place sorting algorithm that repeatedly passes through a list, compares adjacent items and swaps them if they are in the wrong order. Over repeated passes the largest (or smallest) elements "bubble" to their final positions.

ALGORITHM

Start with the full array.

Compare the first and second elements. If they are out of order (depending on ascending/descending), swap them.

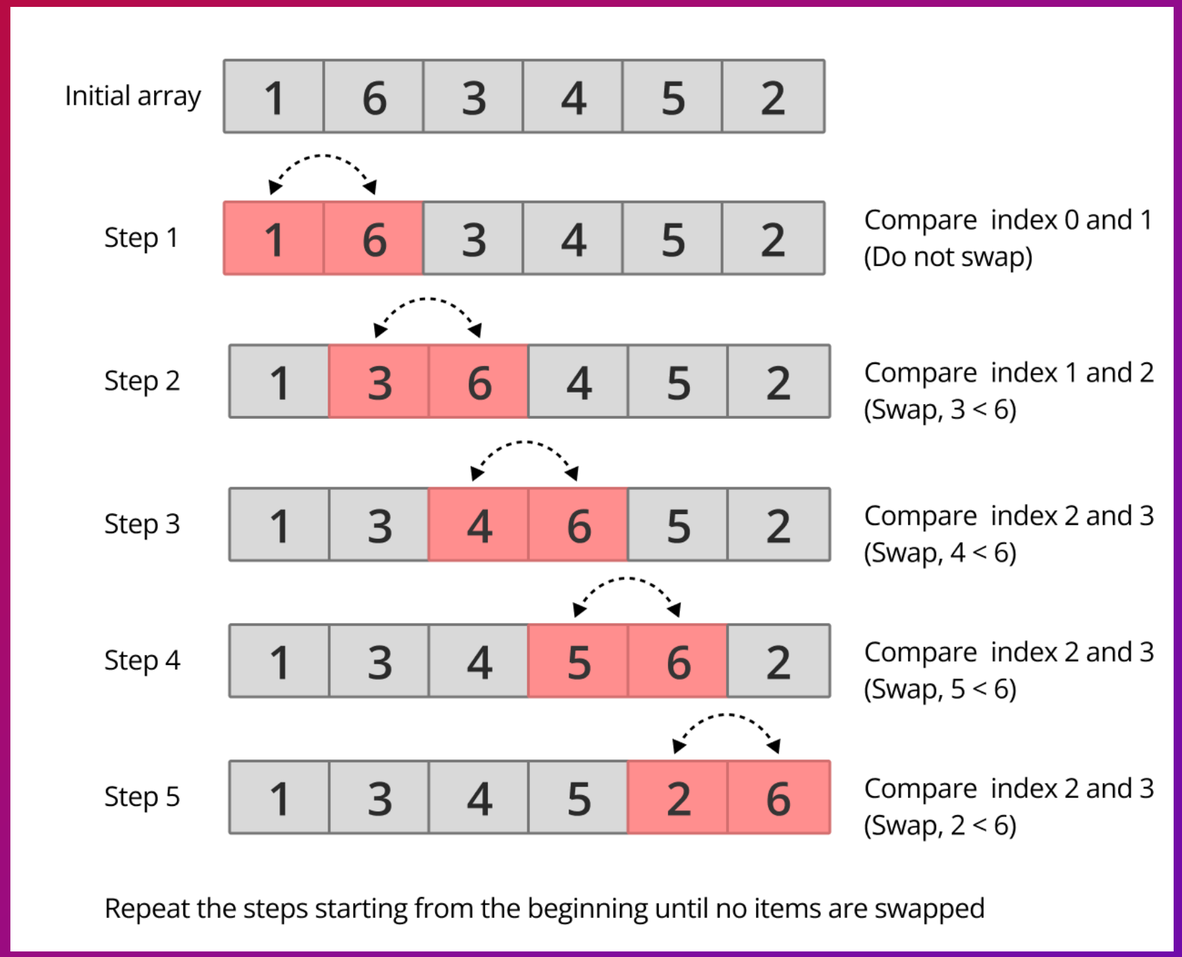
Move one position right and compare the next adjacent pair; swap if needed.

Continue steps 2–3 until you reach the end of the array — after this first pass the largest element will be at the end.

Repeat passes over the array, but each time ignore the last element(s) that are already sorted (reduce the effective length by 1).

Optionally: if a full pass makes no swaps, the array is already sorted — stop early (optimization).

BUBBLE SORT



PSEUDOCODE :

```
bubbleSort(A[0..n-1])
  read Array← from user
    for i from 0 to n-2 do
        if A[i] > A[i+1] then
            swap A[i] and A[i+1]
                swapped ← true
        end if
    end for
    n ← n - 1
  until not swapped
end
```

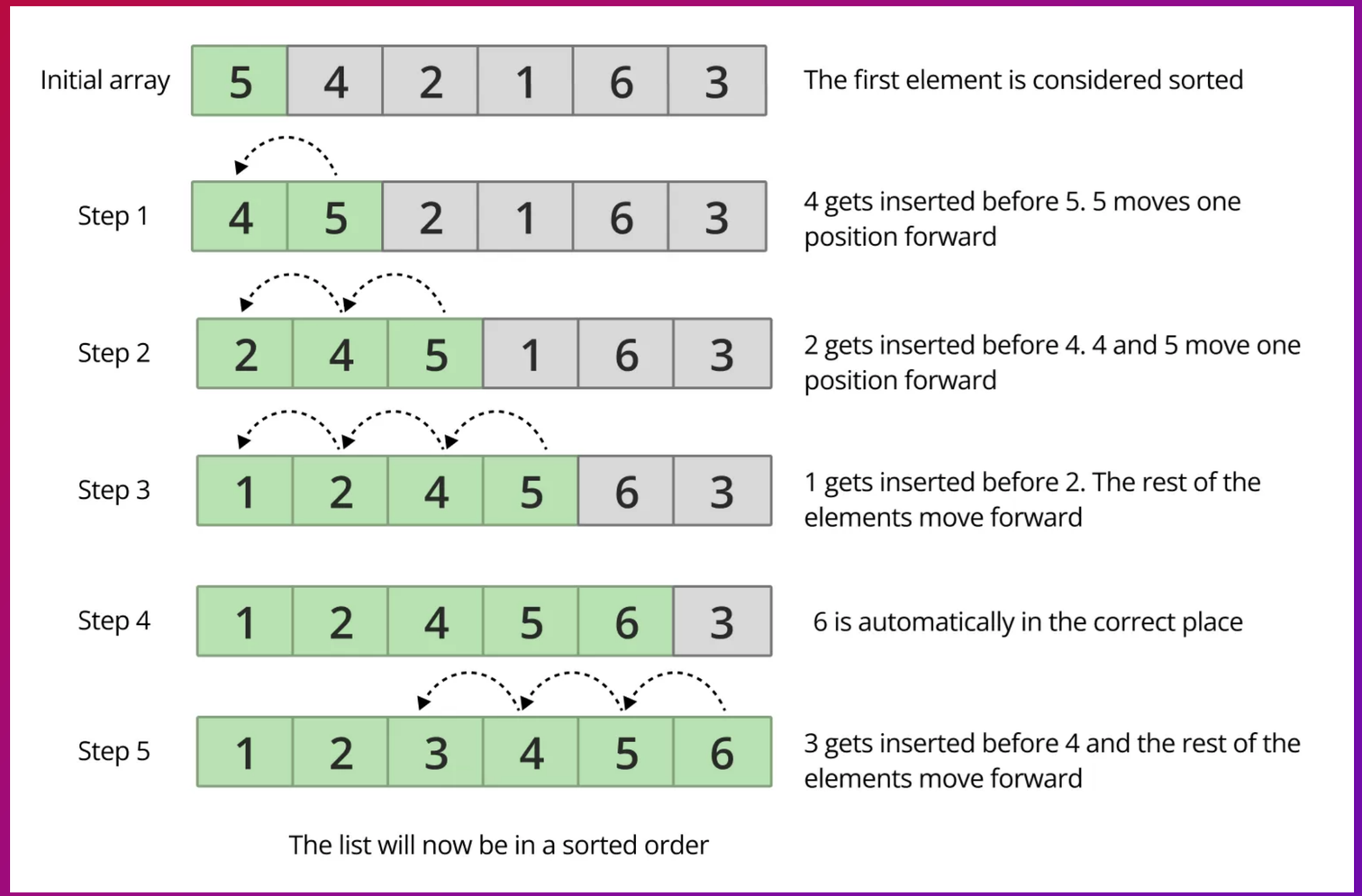
INSERTION SORT

Insertion Sort is a simple sorting algorithm that works the same way people arrange playing cards in their hands. It builds the final sorted array one element at a time by picking each element and placing it in its correct position relative to the already sorted part. It is efficient for small datasets or when the list is already partially sorted, but not suitable for very large datasets compared to advanced algorithms like Quick Sort or Merge Sort.

ALGORITHM

- Start with the second element (index 1) because a single element (index 0) is already sorted.
- Take the current element (key) and compare it with elements in the sorted part (on the left).
- Shift all elements that are greater than the key to one position right.
- Insert the key at its correct position.
- Repeat this for all elements until the array is fully sorted.

INSERTION SORT



PSEUDOCODE :

```
InsertionSort(A, n):
  for i = 1 to n-1 do
      key = A[i]
      j = i - 1

      while j >= 0 and A[j] > key do
          A[j + 1] = A[j]
          j = j - 1
      end while

      A[j + 1] = key
  end for
```

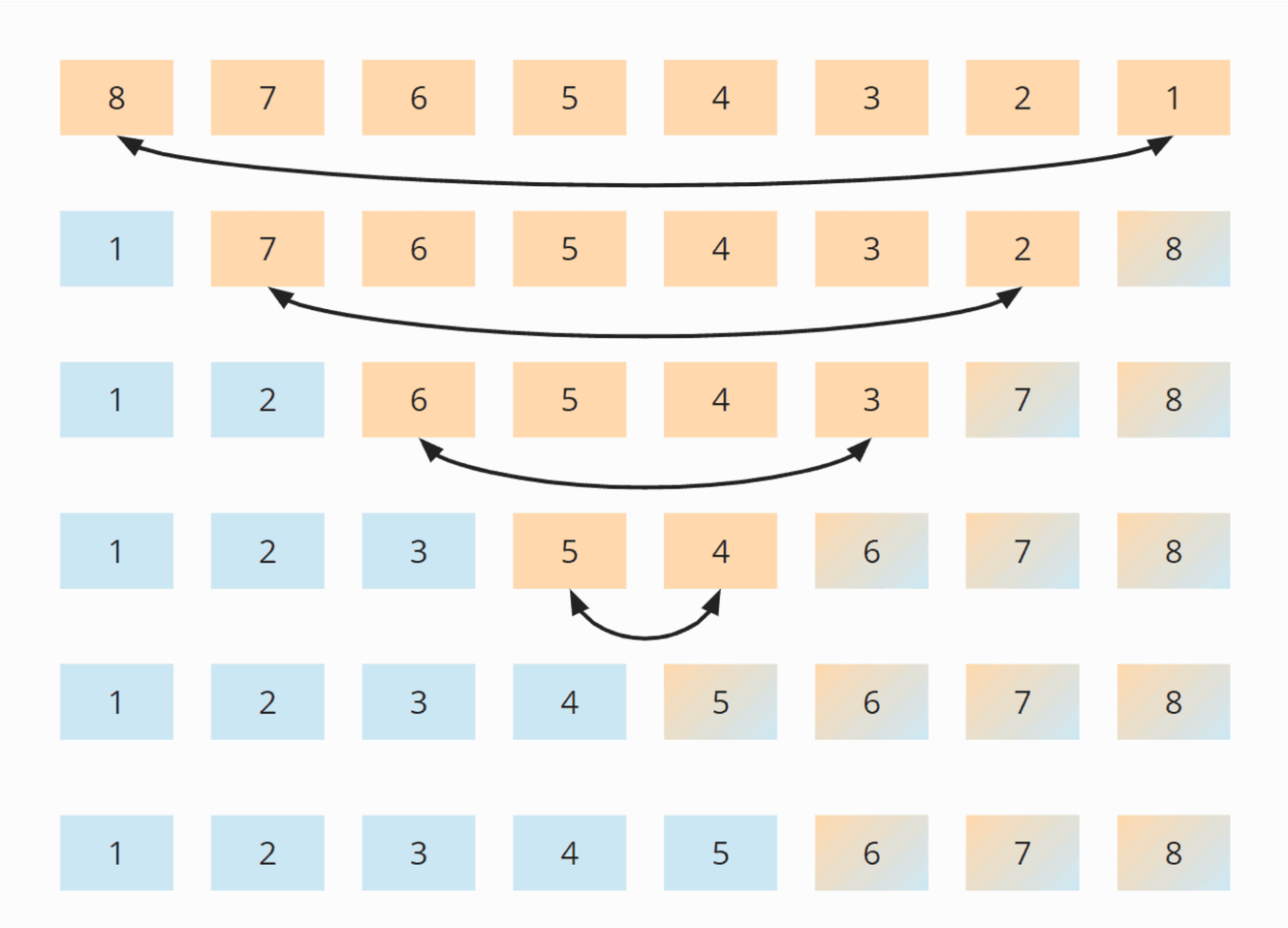
SELECTION SORT

Selection Sort is a simple comparison-based sorting algorithm. It works by dividing the array into two parts: a sorted part (on the left) and an unsorted part (on the right). In each step, the smallest (or largest, depending on order) element from the unsorted part is selected and swapped with the first unsorted element. This process continues until the entire array is sorted.

ALGORITHM

- Start with the first element (index 0).
- Find the smallest element in the unsorted part of the array.
- Swap this smallest element with the first element of the unsorted part.
- Move the boundary between sorted and unsorted part one step forward.
- Repeat the process until the whole array is sorted.

SELECTION SORT



PSEUDOCODE :

```
SelectionSort(A, n):
for i = 0 to n-2 do
    minIndex = i

    for j = i+1 to n-1 do
        if A[j] < A[minIndex] then
            minIndex = j
        end if
    end for

    if minIndex != i then
        swap A[i] and A[minIndex]
    end if
end for
```

LONGEST COMMON SUBSEQUENCE

Given two strings, L1 and L2, the task is to find the length of the Longest Common Subsequence. If there is no common subsequence, return 0. A subsequence is a string generated from the original string by deleting 0 or more characters, without changing the relative order of the remaining characters.

For example, subsequences of "ABC" are "", "A", "B", "C", "AB", "AC", "BC" and "ABC". In general, a string of length n has $2n$ subsequence.

1. **Read two strings** from the user, call them `X` and `Y`.
2. **Find their lengths** `m` and `n`.
3. **Create a table** `L[m+1][n+1]` and set all values to `0`.
4. **Fill the table**:
   - For each character in `X` (i = 1 to m):
     - For each character in `Y` (j = 1 to n):
       - If `X[i-1] == Y[j-1]`, then `L[i][j] = L[i-1][j-1] + 1`
       - Otherwise, `L[i][j] = max(L[i-1][j], L[i][j-1])`
5. **The LCS length** is `L[m][n]`.
6. **Find the LCS string** by starting from `L[m][n]` and moving backwards:
   - If characters match, add it to the LCS string and move diagonally up-left.
   - If not, move to the cell with the larger value (up or left).
7. **Print the LCS length** and the LCS string.

LONGEST COMMON SUBSEQUENCE

```
Enter first string: ABCBDAB
Enter second string: BDCAB
Length of LCS: 4
LCS: BCAB
```

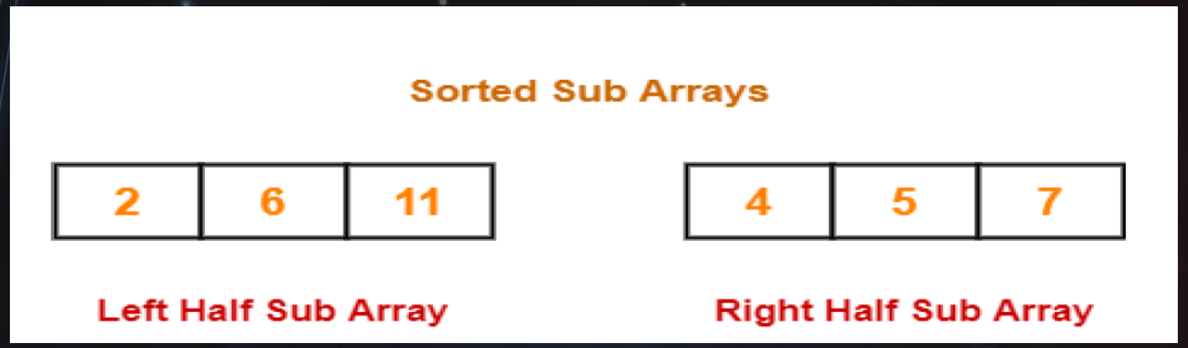
MERGE SORT

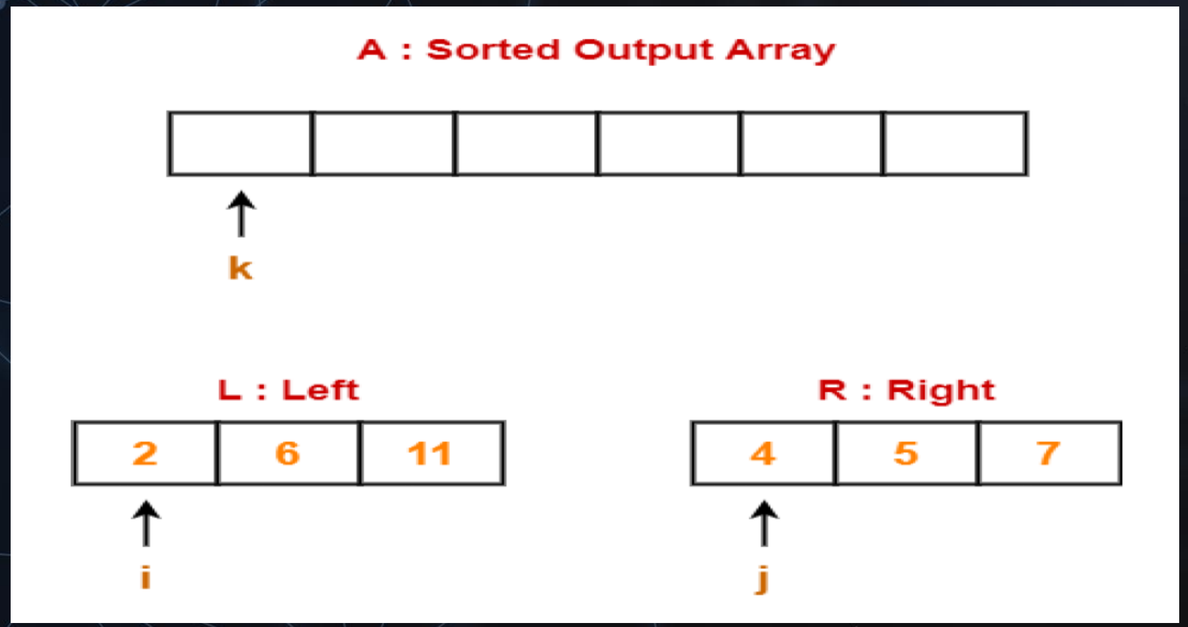
Merge Sort is a divide-and-conquer sorting algorithm. It divides the array into two halves, sorts each half, and then merges the two sorted halves to produce a fully sorted array. The main idea is breaking a big problem (sorting the whole array) into smaller problems (sorting smaller subarrays). Merge Sort is very efficient and guarantees a time complexity of O(n log n) in all cases—best, average, and worst. It is also a stable sorting algorithm, meaning it preserves the original order of equal elements.

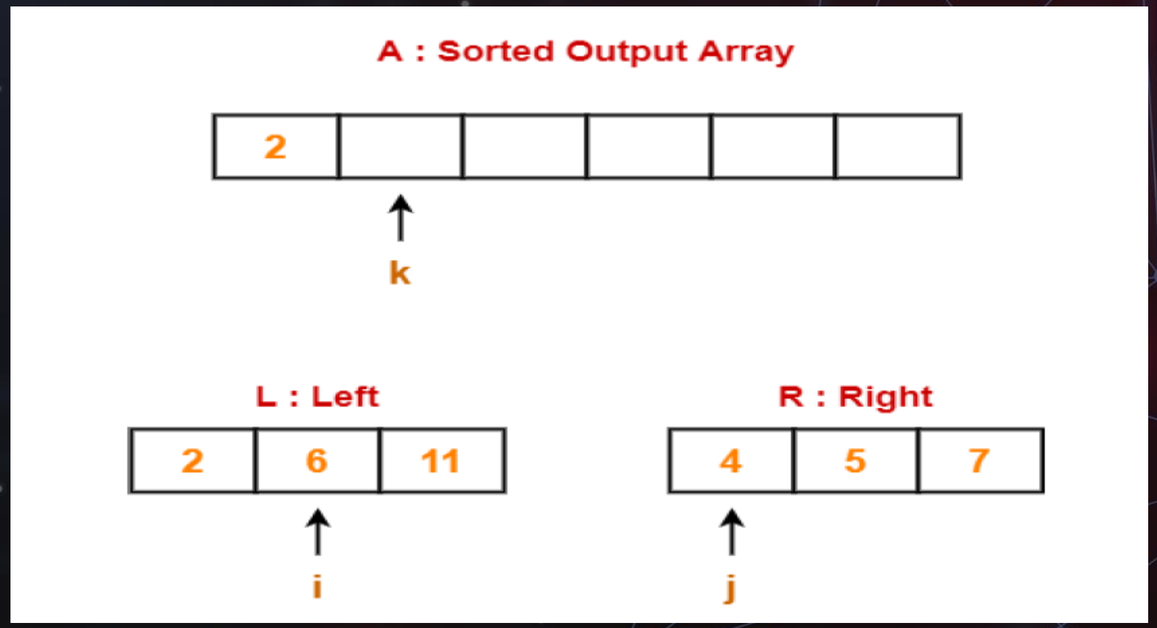
ALGORITHM

- Divide the array into two halves (left and right).
- Recursively sort the left half.
- Recursively sort the right half.
- Merge the two sorted halves into one sorted array.
- Repeat until the entire array becomes sorted.

MERGE SORT


Sorted Sub Arrays
2
6
11
4
5
7
Left Half Sub Array
Right Half Sub Array


A : Sorted Output Array
k
L : Left
2
6
11
i
R : Right
4
5
7
j


A : Sorted Output Array
2
k
L : Left
2
6
11
i
R : Right
4
5
7
j

ANY QUESTION ?

GOOGLE IT / ASK CHATGPT

THNAK YOU

Password</Text>
this.setState({password})}
s.button} onPress={this.handleLogin}>
fontWeight: "500" }}>Sign In</Text>
gnSelf: "center", marginTop: 32}}>
fontSize: 13}}>
style={{ fontWeight: "500", color: "#E9446A" }} onPress={() => this.props.navigation.navigate
Ln 52, Col 60
Spaces: 2
UTF-8
LF
JavaScript
Go Live
MacBook Pro